



Week 9

✓ Interprocess Communication (IPC)

- Signals
- Exchange Data (Msg passing, Share memory)
- Unix Inter Process Communications
 - Signal
 - Handling Signal
 - handling Multiple Signal Types
 - Sending a Signal: ANSI C Function raise()
 - Sending a Signal to Another Process: System Call kill()

| Content

✓ Interprocess Communication (IPC)

- What do we mean by Interprocess Communication?
 - Communication between 2 or more processes

Interprocess Communication

- Processes within a system may be **independent** or **cooperating**
 - Cooperating process can affect or be affected by other processes, including sharing data
 - Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Cooperating processes need **interprocess communication (IPC)**
 - Three models of IPC
 - Signal
 - Shared memory
 - Message passing
-
- at the same time, there can be more than 1 process running in the system.
 - but if they work alone without need to exchange information → don't have to worry about interprocess communication.
 - the process especially like related process like parent and child always need to communicate with each other.
 - That's why we have to consider this (or os just provide a way to allow process to communicate)
 - Reasons for process communicate (cooperating processes)
 - Sharing information
 - Computation speedup (like if you want to find prime number between 1 and 1 million → you can create many thread or process, find in each range → combine)
 - Modularity
 - The mechanism that OS provide for process to communicate is call **IPC(interprocess communication)**
 - 3 major ways that OS provide for IPC (mechanism)
 1. Signal

2. Shared memory
3. Message passing

Signals

(Signal is one of IPC, but it doesn't use to exchange data)

Signals

- Software interrupts that notify a process that an event has occurred
 - Do not allow processes to specify data to exchange with other processes
 - Processes may catch, use OS default, or mask a signal
 - Catching a signal involves specifying a routine that the OS calls when it delivers the signal
 - Use operating system's default action to handle the signal
 - Masking a signal instructs the OS to not deliver signals of that type until the process clears the signal mask
- signals is a software interrupts used when the process need to notify other process that some event occur, or user might need to send event to the process.
- **No data exchange between process.**
- Signal is not use to exchange data, it use to notify.
- when a process receive a signal, actually, there are 3 ways that process can choose to work with the signal
 1. **Catch** - a process has a function to manage (work with) that signal.
 - exception handling (java) → you can think of try block, when error occur send signal and catch block catch it.
 - in your program code, you have code handling that signal (this idea)
 - you have signal handling code to work with the signal.
 - you write signal handler code to work with the signal that get send to the process
 - Signal is form interrupt → instead of allow OS to do default task → you write program to catch signal yourself.
 2. **OS default (like i don't care, just let OS handle the signal.)**

- OS default action to that signal
- Let OS manage the signal (don't have to write signal handling code)
- when to type ctrl+c, it mean stop, terminate (default)

3. **Masking** (choose to tell os to delay the signal)

- Mask mean you said that "hey, at this moment, i don't want to get this signal"
- At this point, i like to delay working with this signal. → maybe because your program is working with other signal (like if might said if any signal come, i want to delay for a while, while i working with former signal, after that i'll come back work with the signal)

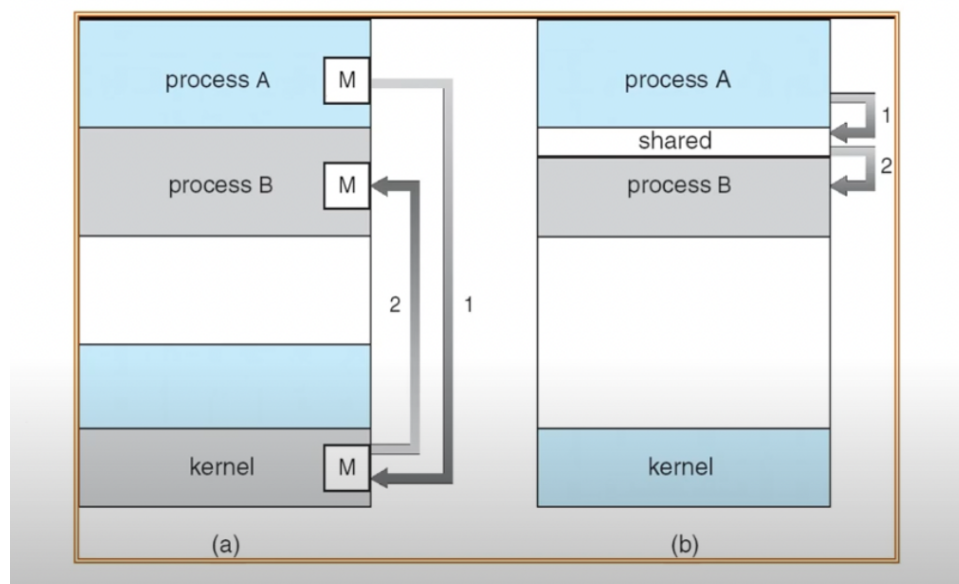
Actually, for some signal, you don't have choice, you must follow OS default, you can't catch or mask anything.

Exchange Data (Msg passing, Share memory)

Communication Model

if you want to exchange data between processes, there are 2 ways you can choose

Communications Models



(a) Message Passing

- for message passing, it just like you ask OS to create a mailbox(msg box). (like Msg box in kernel).
- when process a want to send data to process b, process a then write data in the kernel message box, then process b can read data from this as well. (msg box → tu phak jodmai)
- advantage? → because the kernel mode → switch back and forth when message passing(kernel mode, user mode), a little slow.
- Do you think can you have message box like large → OS may said to have some limitation.
 - if the size is limited to 1000 bytes, does it mean that we can't send data larger than that?
 - No, if the process a want to send the data that larger than message box size → divide the data.
 - Let's said process a divide data into 3 chunks
 - it send the first chuck to kernel msg box, but if process B haven't read the first chunk yet, OS won't allow process a to write 2nd chuck → if it allowed, first chunk will be replaced
 - The idea is OS does the synchronization → the writer write and wait to read data first
- Benefit → OS can provide the synchronization mechanism for us.
- Disadvantage → size of msg box may not be large enough (you may have to divide data)
 - can set msg box size to be very large
 - kernel mode → slow (switch)



writing program that communicate to each other over network → is kind of message passing. OS handle for you (socket) → network concept (divide data when transport)

(b) Shared Memory

- Each process has it own memory area → other process can't interfere to the memory area of another process.

- Process A tell OS that I want a memory area and i want it to be share with other process
 - if process A said that, OS will allow process B to access
 - process A can write data, process B can read
- Shared memory is not in kernel → therefore, size of memory you want can be larger than in a kernel mode here
- pretty easy, doesn't require OS task to help you to manage, but for synchronization, programmer have to do the synchronization themself. (get larger area, but manage synchronization yourself)



In network, shared memory is very hard to do (you need some tool, special program to kind of give illusion to computer system that we have global memory)

Unix Inter Process Communication

3 Mechanism

1. Signal
2. Pipe (1 way to do message passing)
3. Shared memory

Signals

Ex of source of event that can be send to the proces, generating signal

What Are Signals?

- Signals are generated when an event occurs that requires attention.
- Signals can be generated from the following sources:
 - Hardware
 - Example: division by zero, address protection
 - Other Processes
 - Example: a child process notifying its parent process that it has terminated.
 - User
 - Example: pressing a keyboard sequences that generate a quit, interrupt or stop signal.

read in here too

- Signals are software interrupt, source of event

Example source of event that has send to the process

- if your code, program contain statement that divide by 0
 - the one who generate the event → cpu(hardware), but the cause is your program, so it still an software interrupt.
- User press ctrl+c

Signal (Contd.)

- Different systems define different signals. The available signals and their names are defined in the header file
- signal.h
- Note that signal.h is a header file of ANSI C. However, not all signals listed below are supported by the ANSI C standard.
- Under Unix, all signal names start with SIG. The following are some examples:

For Unix based system, we define function and constant inside signal.h file

- interrupt is integer, signal is also integer (this is define in signal.h)
- instead of remember 0 is this signal, 1 is this signal → CONSTANT define in signal.h

Signal (Cont.)

- SIGFPE An arithmetic exception (e.g., division by zero, overflow and underflow) has occurred.
 - SIGINT An interrupt (e.g., Ctrl-C) has occurred.
 - SIGTERM Software termination.
 - SIGQUIT (non-ANSI C) Quit execution (e.g., Ctrl-\).
 - SIGUSR1 and SIGUSR2 (e.g., non-ANSI C) User-defined signal 1 and 2.
 - SIGKILL (non-ANSI C) Kill the process (cannot be caught or ignored)
- SIGFPE → floating point error

- for arithmetic exception (divide by zero, overflow)
- **SIGINT → interrupt**
 - an interrupt like when ctrl+c is press + signint will be send
- **SIGTERM → software termination (same)**
 - for example when parent want to terminate its child, it send SIGTERM to the child
- **SIGQUIT (not standard) (same)**
 - in some system it work
 - like you may press ctrl-\
- **SIGUSR1, SIGUSR2 → user-defined**
 - if your program/process want to send the signal yourself (learn later)
- **SIGKILL → kill your process, when the process receive → it has no choice → can't catch, it must do what OS want you to do (you have to terminate) (same)**
 - example of signal that when the process receive, it has no choice, it has to do whatever os need it to do, must terminate → can't catch SIGKILL.



Handling Signal

Handling Signals: Function signal()

- The most important function for handling signals is signal().
- signal() is used to register a signal handler function.

Signal()

- signal function is not used to handling signal → it used to register a signal handler function.

- when your process need to catch signal → you have to call signal function to state that, if this signal occur, please call my function
- signal function require 2 argument
 - first one is integer (which must be signal name or a predefined signal related constant
 - second one is a signal handler function that must be in void (*)(int)
 - accepts an integer arguments and returns nothing
 - you can name this function whatever you want

If the system detects the signal specified in the first argument, the signal handler as specified in the second argument is called.

catch SIGINT

- when sigint is send, call my INThandler function (inthandler must be in this format)

```
#include <stdio.h>
#include <signal.h>
void INThandler(int); /* Ctrl-C handler */
void main(void)
{
    signal(SIGINT, INThandler); /* install Ctrl-C handler */
    while (1) /* loop forever and wait */
        pause(); /* for Ctrl-C to come */
}

void INThandler(int sig)
{
    char c;

    signal(sig, SIG_IGN); /* disable Ctrl-C */
    printf("OUCH, did you hit Ctrl-C?\n" /* print something */
           "Do you really want to quit? [y/n] ");
    c = getchar(); /* read an input character */
    if (c == 'y' || c == 'Y') /* if it is y or Y, then */
        exit(0); /* exit. Otherwise, */
    else
        signal(SIGINT, INThandler); /* reinstall the handler */
}
```

- at first comment signal(SIGINT, INThandler) in main

```

sarun@sarun-flowx13:~/os_course/signal$ code sig-0.c
sarun@sarun-flowx13:~/os_course/signal$ gcc -o sig-0 sig-0.c
sarun@sarun-flowx13:~/os_course/signal$ ./sig-0
^C
sarun@sarun-flowx13:~/os_course/signal$ |

```

- not commenting out

```

sarun@sarun-flowx13:~/os_course/signal$ gcc -o sig-0 sig-0.c
sarun@sarun-flowx13:~/os_course/signal$ ./sig-0
^COUCH, did you hit Ctrl-C?
Do you really want to quit? [y/n] |

```

- in this main program, we register when SIGINT occur → call my function in INThandler, os will call handler and pass this signal to function
- SIG_IGN → ignore → mask the signal
- signal(sig, SIG_IGN) → i don't want to get ctrl+c again, because i working with ctrl+c. (sometime, user press many time, 1 time is enough, i don't want to get ctrl+c anymore)
- if else
 - if user doesn't press y, you should reinstall the handler
 - depend on OS as well (reinstall) → but to be sure, do it yourself.



Handling Multiple Signal Types

Separate each function for 1 signal

Handling Multiple Signal Types

- There are two choices:
- Install a signal handler for each signal.
- `signal(SIGINT, INThandler);`
- `signal(SIGQUIT, QUIThandler);`
- `void INThandler(int sig)`
- {
-
- }
- `void QUIThandler(int sig)`
- {
-
- }

- use this idea

Write 1 function to handle many signal

- but actually, you can write 1 function to handle multiple signal using switch, if else

Handling Multiple Signal Types (Contd.)

- Write a single handler in which its only argument is used to determine what signal is sent in:
- `signal(SIGINT, SIGhandler);`
- `signal(SIGALRM, SIGhandler);`
- `void SIGhandler(int sig)`
- {
-
- `switch (sig) {`
- `case SIGINT : .....`
- `case SIGALRM : .....`
- `default : .....`
- }
-

Which one is better? (separate to each function or one function)

- It depends
 - if the handle function is simple → 1 function multiple signal is better
 - if you have to do complicate thing to handle the signal → separate is better
 - because disadvantage of separate function → remember lot of function name, but you can choose.



Sending a Signal: ANSI C Function: raise()

(what if a process want to send a signal)

Sending a Signal: ANSI C Function raise()

- ANSI C has a library function `raise()` for sending a signal to the program that contains this `raise()` call. The following is the function prototype:
 - `int raise(int sig)`
 - If this function is executed successfully, the signal specified in `sig` is generated.
 - If the program has a signal handler, this signal will be processed by the that signal handler.
 - Otherwise, this signal will be handled by the default way. If the call is unsuccessful, `raise()` returns a non-zero value.
 - Note that function `raise()` can only send a signal to the program that contains it. `raise()` cannot send a signal to other processes.
 - To send a signal to other processes, the system call `kill()` should be used.
- You can use `raise()`, but `raise` is use when a process need to send event(signal) to itself. (like try, catch block) something error in the program
 - how `raise` work, when do we have to use `raise`.

Example

raise() Example (2)

```

void main(void)
{
    long fact;

    printf("Factorial Computation:\n\n");
    signal(SIGUSR1, SIGHandler); /* install SIGUSR1 handler */
    for (prev_fact = i = 1; ; i++, prev_fact = fact) {
        fact = prev_fact * i; /* computing factorial */
        if (fact < 0) /* if the results wraps around */
            raise(SIGUSR1); /* we have overflow, print it */
        else if (i % 3 == 0) /* otherwise, print the value */
            printf(" %ld! = %ld\n", i, fact);
    }
}

```

```
sarun@sarun-flowxl3:~/os_course/signal$ ./raise
Factorial Computation:
1! = 1 (1)
2! = 2 (1)
3! = 6 (2)
4! = 24 (6)
5! = 120 (24)
6! = 720 (120)
7! = 5040 (720)
8! = 40320 (5040)
9! = 362880 (40320)
10! = 3628800 (362880)
11! = 39916800 (3628800)
12! = 479001600 (39916800)
13! = 6227020800 (479001600)
14! = 87178291200 (6227020800)
15! = 1307674368000 (87178291200)
16! = 20922789888000 (1307674368000)
17! = 355687428096000 (20922789888000)
18! = 6402373705728000 (355687428096000)
19! = 121645100408832000 (6402373705728000)
20! = 2432902008176640000 (121645100408832000)

Received a SIGUSR1. The answer is 20! = 2432902008176640000
sarun@sarun-flowxl3:~/os_course/signal$ |
```

write program to test what is the maximum factorial number that the system can handle

- overflow idea → if it positive sign of integer is too large, it will go back to negative sign.
- find factorial in for loop, and if factorial is less than 0 → overflow occur → raise(SIGUSR1)
 - if SIGUSR1 occur → call SIGhandler

Sending a Signal to Another Process: System Call kill()

Sending a Signal to Another Process: System Call kill()

- To send a signal to another process, we need to use the Unix system call(). The following is the prototype of kill():
- `int kill(pid_t pid, int sig)`
- System call kill() takes two arguments.
 - The first, pid, is the process ID you want to send a signal to,
 - and the second, sig, is the signal you want to send. Therefore, you have to find some way to know the process ID of the other party.
- If the call to kill() is successful, it returns 0; otherwise, the returned value is negative.
- Because of this capability, kill() can also be considered as a communication mechanism among processes with signals SIGUSR1 and SIGUSR2.

- kill require 2 argument
 - process id that you want to send signal to
 - signal itself

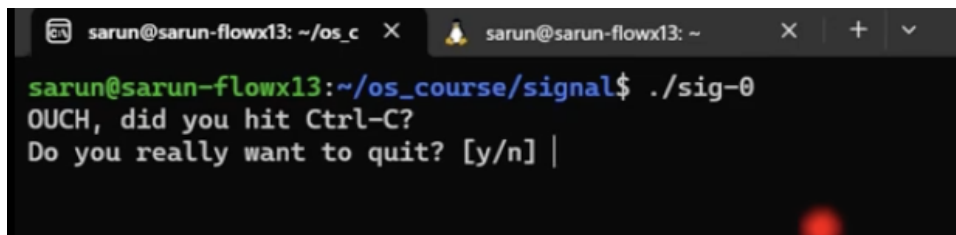
- why the system call is kill?
 - it come from the command kill, the command kill in the OS mean kill is use to send any signal to the process. But by default, if you don't specify the signal → SIGTERM is set.

Example

- run sig-0, open other terminal do ps-a

```
sarun@sarun-flowx13:~$ ps -a
PID TTY          TIME CMD
123 pts/1        00:00:00 sh
128 pts/1        00:00:00 sh
132 pts/1        00:00:03 node
180 pts/1        00:00:16 node
1890 pts/0        00:00:00 sig-0
1930 pts/4        00:00:00 ps
sarun@sarun-flowx13:~$ kill -SIGINT 1890
```

- you can also use only kill -INT (process id)



```
sarun@sarun-flowx13: ~/os_c  X  sarun@sarun-flowx13: ~  X  +  v
sarun@sarun-flowx13:~/os_course/signal$ ./sig-0
OUCH, did you hit Ctrl-C?
Do you really want to quit? [y/n] |
```

Modify and handle SIGTERM as well

```
19 void main(void)
20 {
21     signal(SIGINT, INThandler);
22     signal(SIGTERM, INThandler);           /* install Ctrl-C handler
23     while (1)                             /* loop forever and wait
24         pause();                          /* for Ctrl-C to come
25 }
26
```

```
sarun@sarun-flowx13:~$ ps -a
PID TTY          TIME CMD
123 pts/1        00:00:00 sh
128 pts/1        00:00:00 sh
132 pts/1        00:00:03 node
180 pts/1        00:00:17 node
2080 pts/0        00:00:00 sig-0
2094 pts/4        00:00:00 ps
sarun@sarun-flowx13:~$ kill 2080
```

- not specify signal to send → by default sigterm is send

```
sarun@sarun-flowx13: ~/os_c
sarun@sarun-flowx13: ~
sarun@sarun-flowx13:~/os_course/signal$ ./sig-0
OUCH, did you hit Ctrl-C?
Do you really want to quit? [y/n] y
sarun@sarun-flowx13:~/os_course/signal$ gcc -o sig-0 sig-0.c
sarun@sarun-flowx13:~/os_course/signal$ ./sig-0
OUCH, did you hit Ctrl-C?
Do you really want to quit? [y/n] |
```

Example to show SIGQUIT can't be catch

```
19 void main(void)
20 {
21     signal(SIGINT, INThandler);
22     signal(SIGTERM, INThandler);
23     signal(SIGKILL, INThandler); /* install Ctrl-C handler
24     while (1) /* loop forever and wait
25         pause(); /* for Ctrl-C to come
26 }
```

```
sarun@sarun-flowx13:~$ ps -a
PID TTY          TIME CMD
123 pts/1        00:00:00 sh
128 pts/1        00:00:00 sh
132 pts/1        00:00:03 node
180 pts/1        00:00:19 node
2294 pts/0        00:00:00 sig-0
2305 pts/4        00:00:00 ps
sarun@sarun-flowx13:~$ kill -KILL 2294
```

Nothing happen

(another way to call sigkill)

```
sarun@sarun-flowx13:~$ kill -9 2375
sarun@sarun-flowx13:~$
```

- 9 means sigkill

You can also use signal function call

Socrative

1. IPC model that does not exchange data between processes is signal.
2. IPC model that allow processes to exchange data and the synchronization is managed by OS is Message Passing.
3. IPC model that allow processes to exchange data but the synchronization is the responsibility of programmer is Shared Memory.
4. Message passing mechanism that is provided by the UNIX OS is pipe.
5. A signal can be generated from Hardware, Processes, or User.
6. In general, when a process receives a signal it can catch that signal, mask that signal, or use OS default for that signal.
7. To catch a signal, a programmer writes a function that will be called when the signal is sent to the process. True
8. There are signals that cannot be caught or disabled. True
9. The signal that will be sent to the process when a user press Ctrl-c is SIGINT.
10. System call signal() is used to send a signal from process to process. False
11. A signal handler function in a UNIX like OS must accept one parameter of type int which is the signal that is sending to the process. True
12. You must disable all signals in a signal handler function. False (it an option, but recommened to do.)
13. To disable the incoming signal in a signal handler, the macro that you will pass as the second parameter to the signal() function is SIG_IGN
14. A process can handle only one signal. False

15. System call `raise()` is used to send a signal from one process to another.

False

16. System call `kill()` is used only to send the signal `SIGKILL` from one process to another. False

(17, 18) Coding

Homework

1. Write the following C program. A parent forks a child process, the parent executes inside a forever loop. The child sleeps for 5 seconds to give time to the parent to enter a forever loop. The child then sends the signal `SIGINT` to the parent. After the parent receives the signal, it prints "Get `SIGINT` from child", exit the forever loop, print "Bye", and terminates.

The example output is as follows:

parent: Waiting for my child to send `SIGINT`

child: Sleeping for 5 seconds

child: sending `SIGINT`

child: Bye

get `SIGINT` from child

parent: Bye

Note: Use system call `kill()` send signal from child to parent. To retrieve the parent

process id use the system call `getppid()`.

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
void terminated(int);
int main(void)
{
    pid_t pid[5];
    int i = 0;
    for (i = 0; i < 5; i++) {
        pid[i] = fork();
        if (pid[i] == 0) {
            signal(SIGTERM, terminated);
```

```

        printf("Child: my id is %d\n", getpid());
        while(1);
    }
}
sleep(2);
for(i = 0; i < 5; i++) {
    kill(pid[i], SIGTERM);
}
while(wait(NULL) != -1);
printf("Parent: I have killed all of my children\n");
return 0;
}

void terminated (int sig)
{
    printf("Child: my id is %d, my parent has just terminated\n", getpid());
    exit(0);
}

```

- Write the following C program. A parent forks 5 children processes. Each child prints its own process id and enters a forever loop. Parent sleeps for 2 seconds to give time to all children to enter their forever loop. Parent then sends SIGTERM to all children. When each child process receives the SIGTERM, it prints its process id, and terminates. After all children have been killed, the parent prints "Parent: I have killed all of my children". The example output is as follows:
 Child: my id is 430
 Child: my id is 431
 Child: my id is 432
 Child: my id is 433
 Child: my id is 434
 Child: my id is 430, my parent has just terminated me
 Child: my id is 434, my parent has just terminated me
 Child: my id is 433, my parent has just terminated me

Child: my id is 432, my parent has just terminated me

Child: my id is 431, my parent has just terminated me

Parent: I have killed all of my children

Note: Use the system call kill() send signal from parent to child. The parent needs an

array to store process id of all children. The system call getppid() is used to get the process

id of the process.

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
void    ding(int);
int done = 1;
int main(void)
{
    signal(SIGINT, ding);
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("child: Sleeping for 5 seconds\n");
        sleep(5);
        printf("child: sending SIGINT\n");
        kill(getppid(), SIGINT);
        printf("child: Bye\n");
    }
    else {
        printf("parent: Waiting for my child to send SIGINT\n");
        while (done); /* loop and wait */
        wait(NULL);
        printf("parent: Bye\n");
    }
    return 0;
}
void ding(int sig)
{
```

```
        printf("get SIGINT from child\n");  
        done = 0;  
  
    }
```